

MedPin readiness audit

Where the platform actually stands against the 46-item release checklist — audited by probing the running code, not by reading it. Two things block a redeploy, and staging now exists to catch the third. Everything below distinguishes what has been *proven* from what merely passes.

BACKEND TESTS 1681 110 files, all passing	ANALYZER 0 errors and warnings	API ROUTES 250 144 of them mutating	ROLES 8 was 4 this morning	UNDEPLOYED 7 commits ahead of <code>flint/main</code>
BLOCKERS 2 both outside the code				

What blocks a redeploy

Both are a console toggle or an environment variable. Neither can be fixed by committing anything.

Applicants cannot confirm their email address

`ADMIN_CONSOLE_URL` is unset, so the confirmation paragraph is omitted from the email — correctly, rather than sending a dead link — and `contactEmailVerified` can never become true for anybody. The server now reports this itself at boot and in the console's attention bell, so it stops being something you have to remember.

YOURS · ONE ENVIRONMENT VARIABLE

Razorpay cannot take the payment methods Indian customers use

Proven against the account API: `upi: false` and international payments off, which is why the test card fails. There is no self-serve API for this — three PATCH attempts returned 404. Enable UPI and UPI Autopay in Dashboard → Settings → Configuration → Payment Methods. International Payments is an application Razorpay reviews, and is probably unwanted for INR-billed Indian SaaS anyway.

YOURS · RAZORPAY DASHBOARD

A correction

One item on the earlier version of this page was wrong, and wrong in the direction that costs an afternoon.

The admin console is not missing its API

I reported `admin.medpin.in` as having no API behind it, on the evidence of one 404 on `/api/v1/health`. It proxies exactly `/api/v1/admin/` and `/api/v1/applications/` — and the console calls exactly those two prefixes and nothing else, checked path by path against the live host and against every API call in `web/src`. The page serves, `/applications/options` returns real data, every admin route correctly 401s.

The narrowness is the better arrangement rather than a half-finished one: `/auth/`, `/doctor/` and `/billing/` are not reachable behind the console's first-party session cookie at all. What was broken was my tool's assumption that every host proxies the whole API. It takes a `--console` surface now and passes 7 of 7 there.

How to read the status

This codebase draws one distinction the checklist does not, and it has earned it: a test that passes and a test that was seen to fail are different facts.

PROVEN

Mutation-tested — the code was broken and the test caught it

BUILT

Works and is tested; not adversarially verified

PARTIAL

Some of it, with a named gap

BLOCKED

Waiting on a credential, a toggle or a server

OPEN

Not started

The checklist

All 46 items, grouped by what they actually belong to.

DYNAMIC UI, ROLES AND PERMISSIONS

PROVEN

Dynamic UI engine

Practice → department → role → permissions → plan → capabilities → screen, resolved server-side and sent as identifiers only.

PROVEN

Department-based UI

Two doctors, one practice, one plan, one role — different departments, different screens sharing one panel deliberately.

PROVEN	Role-based UI Doctor's assistant, lab manager, lab technician, practice manager. Head doctor stays <code>isOwner</code> .
PROVEN	Capability system Type ceiling × plan ceiling × grant × role exclusions, with absence permitting throughout.
PROVEN	Widget registry validated Cross-language contract, duplicate-key guard, capability/permission existence, widget-vs-action collision.
PROVEN	Dashboard composition Four precedence tiers, written down: configured department → role default → department default → general.
PARTIAL	Permission system verified Presets, exclusions and the patient-access allow-list are proven. 128 of 144 mutating routes carry no <code>requirePermission</code> — most legitimately (admin surface, self-service, public signup), perhaps 25 genuinely.
PARTIAL	Dashboard composer Deferred with reason. The API takes <code>widgets / quickActions</code> validated against the registry, and the departments screen shows whether a specialty runs the platform's arrangement or its own — but there is no picker. Configuring one means an API call.
BUILT	Subscription entitlement
BUILT	Admin permissions
CLINICAL WORKFLOWS	
PARTIAL	Laboratory workflow Reports, critical flags and a clinic-wide bench view are real. There is no specimen model — see below.
PARTIAL	Cardiology workflow Real data only: caseload, vitals, alerts, lab reports. No ECG, no risk score — see below.
PROVEN	ECG decision documented
PROVEN	Cardiac risk decision documented
PROVEN	No fake clinical data Enforced by the registry rule: every identifier names an endpoint that already answers.
BUILT	Patient, appointment, consultation, prescription, vitals workflows All pre-existing and covered; none re-verified end-to-end on a device this session.

BUILT	Lab result workflow
BUILT	AI permissions
PARTIAL	AI patient-data access secured Scoped per patient and per department; not adversarially tested for retrieval through the assistant.
NAVIGATION, STATES AND THE APP	
PROVEN	Navigation targets Every <code>push</code> in <code>lib/</code> checked against the router, nesting resolved. Plus the ordering invariant <code>go_router</code> depends on: a static route declared after its <code>:id</code> sibling never runs.
PROVEN	Static analysis clean
PROVEN	Production build Debug APK builds. A signed release build has not been made this session.
PARTIAL	Empty and loading states The panels built this session handle both explicitly — a lab panel shows nothing rather than "no critical results" while a request is in flight. Older screens vary.
OPEN	Error and permission-denied states No systematic pass. A 403 from a tightened route currently surfaces as a generic failure.
PARTIAL	Offline and network failure
PARTIAL	Mobile UI tested Analyzer clean and the APK builds. Nothing has been looked at on a handset — deliberately, since driving the device twice landed in your personal WhatsApp.
PARTIAL	Accessibility Every status carries a word, tap targets respect the text scaler, contrast is ratcheted in the console. No screen-reader pass.
OPEN	Performance tested No measurement of login time, dashboard load, or query plans under a realistic dataset.
PAYMENTS, EMAIL AND ADMIN	
PROVEN	Razorpay webhook HMAC over the raw request bytes with the webhook secret — distinct from the key secret — and an idempotent upsert.

BLOCKED	Razorpay test payment Fails because the account has UPI off and international off, not because of the integration.
BUILT	Subscription activation Backend verification is authoritative; the client's success callback never grants a plan.
BLOCKED	Email links verified Still blocked on the variable, but no longer silent: boot warns by name and the console's attention bell shows it.
PROVEN	Admin console verified 7 of 7 smoke checks against the live host. The narrow proxy is correct and now asserted.
BUILT	Notifications
PROVEN	Audit logging A coverage ratchet walks every route: one that changes data and records nothing fails the build, and each exemption must carry a stated reason.
SECURITY, DATA AND RELEASE	
BLOCKED	PAT rotated
PARTIAL	Security audit Dependencies clean (<code>npm audit : 0</code>). Tenant isolation, webhook signatures and the patient-access allow-list are proven. Admin console 2FA is still unenrolled and remains the weakest point.
PROVEN	Environment variables verified Every subsystem that depends on configuration now reports ready / degraded / off with what breaks — including the two that fail open: a Razorpay webhook secret that is missing (every forged callback accepted) or identical to the API secret (every genuine one rejected).
PARTIAL	Database migrations 33 scripts exist. <code>verifyIsolation.js</code> has still never been run against staging.
PROVEN	Backward compatibility A test pins the founding clinic's home screen component-for-component, because the first draft of the migration would have taken the snapshot chart off it.
PROVEN	Unit and integration tests
PARTIAL	End-to-end and negative tests HTTP-level journeys are covered and negative cases are written for everything touched this session. No device-level E2E.

BUILT

Staging deployment and smoke test

Not yet stood up, but every piece is written: pm2 declaring both apps, both Apache vhosts as files, the env template, a runbook, and `scripts/smoke.mjs` — twelve read-only checks, exit non-zero on failure. A test asserts the two environments cannot share a port, a database, an env file, a working copy or an uploads directory.

PARTIAL

Rollback plan

Documented, and honest about its limit: production is a checkout and a pm2 process, so code rolls back in two commands. Migrations do not — every script in `backend/scripts/` is forward-only with no down, so a `mongodump` before one is the rollback.

OPEN

Feature flags

PROVEN

Staging cannot message a real patient

The one thing that can go badly wrong with a clinical staging box. It runs `NODE_ENV=production` on purpose, so nothing in the process knows it is not the real thing — `DEPLOY_ENV=staging` is what lets it check, and any live MSG91, Firebase or SMTP credential reports degraded, naming every one rather than the first.

PARTIAL

Documentation

This page, plus the architecture arguments that live in the code itself.

Three things that must not be invented

The clinical-safety guardrail, applied. Each of these was in the brief's widget list and none of them exists in the data model.

ECG

No model, no field, no source. A waveform panel would render empty forever, which reads as a clinic with no data rather than a feature nobody built. Needs a data model, a source, storage and a representation before any visualisation.

Cardiac risk

The dangerous one. Framingham, QRISK and ASCVD are validated clinical instruments, not arithmetic — and a cardiologist reading a number labelled "cardiac risk" takes it for one. Needs a named instrument and its required inputs documented before implementation, with the calculation source, input data, calculation date and missing inputs all shown.

Specimen workflow

There is no sample model and no ordering lifecycle. "12 pending, 4 processing" would be four numbers with nothing behind them. `LabReport.reviewedBy` exists and *no route writes it*, so "awaiting review" would count every report forever — a staleness indicator that cannot detect staleness. Needs the Order → Specimen → Collected → Received → Processing → Validated → Completed model built and persisted first.

Recommended order

Sequenced by what unblocks what, not by the checklist's numbering.

01 Clear the two blockers

Neither needs code. Set `ADMIN_CONSOLE_URL` and enable UPI. The first is what stops any practice that registers from confirming its email address.

02 Stand up staging

Roughly forty minutes, following `deploy/STAGING.md`. Database, working copy, env, pm2, Apache, then `node scripts/smoke.mjs https://staging.clinq.flintdeorient.in`. Do not copy production's `.env` — the runbook says why, and the server will tell you if you do.

03 Deploy the seven commits to staging and verify there

The UI engine, the four roles, the navigation checks and the readiness reporting. All additive and backward-compatible — a test pins the existing clinic's screen component-for-component — but nothing has yet run against real data.

04 Run `verifyIsolation.js`, at last

It has existed for weeks with nowhere to run. Tenant isolation is the property the whole multi-practice migration rests on, and it has only ever been proven against an in-memory database.

05 Then production, and smoke-test it

`node scripts/smoke.mjs https://clinq.flintdeorient.in` — twelve read-only checks that exit non-zero. Today it reports 11 of 12, the failure being that the deployed build predates the readiness check.

06 Only then, the remaining verification sweep

Error and permission-denied states first, because this session tightened real guards: a practice manager opening a patient record now gets a 403 and the app shows a generic failure. Then the ~25 clinical mutations that still rest on "is a clinician".

Audited against the running code: live health endpoints probed path by path, the Razorpay account API, `npm audit`, and the full 1681-test suite.

Claims marked *Proven* were verified by breaking the code and confirming the test caught it.